

LEGALEASE — CODING & SOLUTION

KEY CODE SNIPPETS:

1. AI GENERATION (ai_core/gemini_generator.py):

```
class GeminiDocumentGenerator:
```

```
    def __init__(self):
```

```
        api_key = os.getenv("GEMINI_API_KEY")
```

```
        if not api_key:
```

```
            raise ValueError("GEMINI_API_KEY not found.")
```

```
        self.client = genai.Client(api_key=api_key)
```

```
        self.model_name = "gemini-3.8-flash"
```

```
    def generate_document(self, document_type, parties, terms, dates):
```

```
        prompt = f"""Generate a comprehensive legal document.
```

```
Document Type: {document_type}
```

```
Parties: {parties}
```

```
Effective Date: {dates}
```

```
Terms: {terms}
```

```
Requirements: Formal language, numbered sections, signature blocks."""
```

```
        response = self.client.models.generate_content(
```

```
            model=self.model_name, contents=prompt)
```

```
        return response.text.strip()
```

2. API ROUTE (legalEaseAPI/routes.py):

```

@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type, request.parties,
        request.terms, request.dates)
    return {"document": response}

```

3. STREAMLIT FRONTEND (frontend/app.py):

```

if st.button("Generate Document"):
    response = requests.post(
        "http://localhost:8000/generate",
        json={"document_type": doc_type, "parties": parties,
            "terms": terms, "dates": dates}, timeout=120)
    st.session_state["generated_text"] = response.json()["document"]

```

4. DOCX EXPORT:

```

def format_docx(text, title):
    doc = Document()
    doc.styles["Normal"].font.name = "Times New Roman"
    h = doc.add_heading(title, level=0)
    for para in text.split("\n"):
        if para.strip():
            doc.add_paragraph(para)

```

```
buf = BytesIO()
```

```
doc.save(buf)
```

```
return buf.getvalue()
```